

SmartRegexValidator: Code Architecture Documentation

1. System Overview

This document outlines the internal modular architecture of the Interactive DFA-Based Email & Password Validation System. The codebase is strictly separated into UI components, theoretical backend engines, and developer utilities to ensure maximum performance, maintainability, and alignment with Automata Theory.

2. Core Modules & Key Code Snippets

A. src/main.py

Role: Application Entry Point.

Logic: Bootstraps the PyQt6 **QApplication**, initializes the main window, and loads the primary **SimulatorPage**.

B. src/automata/email_dfa.py & password_dfa.py

Role: The theoretical backend execution engines.

Logic: Rather than utilizing standard Regex matching, these files contain hardcoded transition dictionaries representing the formal delta (δ) transition functions of a Deterministic Finite Automaton. They parse the user's input string character-by-character and return a comprehensive execution log.

Key Code Snippet (The Transition Function):

```
# This directly represents your TOA Transition Table
transitions = {
    'q0': {'letter': 'q1'},          # Start state expects a letter/number
    'q1': {'letter': 'q1', '@': 'q2'}, # Reading username, waiting for @
    'q2': {'letter': 'q3'},          # Read @, expects domain letter
    'q3': {'letter': 'q3', '.': 'q4'}, # Reading domain, waiting for dot
    'q4': {'letter': 'q5'},          # Read dot, expects extension letter
    'q5': {'letter': 'q5'}           # Final Accept state (e.g., "com")
}
```

Explanation: This dictionary lookup ensures $O(1)$ constant-time transitions, guaranteeing the strict deterministic nature of the machine without NFA backtracking.

C. src/gui/simulator.py

Role: Frontend execution and animation handler.

Logic: This class handles the **QTimer** threading to animate the machine without freezing the interface. The most critical function is **update_ui_for_step**, which visually maps the current active state to the UI.

Key Code Snippet (Dynamic Image Loading):

```
# 2. Update Tape & Image
if step_idx == 0:
    self.update_tape(-1)
    # ADDED "diagrams" to the path here:
    img_path = os.path.join(BASE_DIR, "assets", "diagrams", f"{self.current_prefix}_base.png")
    self.status_label.setText("Machine Halted. Ready for Input.")
    self.status_label.setStyleSheet("color: #020100; font-size: 18px; font-weight: bold;")
else:
    self.update_tape(step_idx - 1)
    t_state = self.current_log[step_idx - 1]['to_state']
    safe_name = t_state.replace(" / ", "_").replace(" ", "_")
    # ADDED "diagrams" to the path here:
    img_path = os.path.join(BASE_DIR, "assets", "diagrams", f"{self.current_prefix}_{safe_name}.png")
```

Explanation: This block dynamically constructs the file path to the correct Graphviz node image based on the backend state, providing a zero-lag "flipbook" animation.

D. src/utils/graph_generator.py

Role: Offline developer utility.

Logic: Utilizes the Graphviz library to render 300 DPI high-resolution state machine diagrams. It utilizes a pre-rendering strategy to build an image for every possible state highlight before the application even runs.

3. Execution Data Flow

1. **Input:** User provides a string via the PyQt6 GUI.
2. **Processing:** The GUI passes the string to the selected automata backend.
3. **Execution:** The backend processes the strict transition dictionary and returns a full mathematical trace log.
4. **Rendering:** The GUI QTimer iterates through the log, matching the to_state to the corresponding pre-rendered image in the assets/diagrams/ folder.
5. **Output:** The UI updates the Turing Tape, Live Log, and Image Canvas simultaneously until the machine halts.